

CS-306-MJ-P — PRACTICAL EXAM SLIP 1

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a Java program to print the sum of elements of the array. Also display array elements in ascending order.

Aim:

To calculate the sum of array elements and display them in ascending order.

Theory:

Arrays store values of the same type. Traversal calculates the sum and Arrays.sort() arranges integer values in ascending order.

Algorithm / Steps:

- Read n and array elements.
- Add each element to sum.
- Sort the array.
- Display sum and sorted array.

Program:

```
import java.util.*;
class ArraySumSort{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in); int n=sc.nextInt(),sum=0;
        int[] a=new int[n];
        for(int i=0;i<n;i++){a[i]=sc.nextInt();sum+=a[i];}
        Arrays.sort(a);
        System.out.println("Sum = "+sum);
        System.out.println("Ascending order: "+Arrays.toString(a));
    }
}
```

Expected Output:

```
Input: 5 → 30 10 50 20 40
Sum = 150
Ascending order: [10, 20, 30, 40, 50]
```

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Design a College/School Registration Form using HTML5 and CSS3 and apply appropriate HTML5 validation attributes.

Aim:

To design a validated College/School Registration Form using HTML5 and CSS3.

Theory:

HTML5 provides semantic form controls and validation attributes such as required, pattern, minlength, maxlength, email and date.

Algorithm / Steps:

- Create the form.
- Add suitable input types.
- Apply validation attributes.
- Style the form with CSS3.
- Test invalid input in the browser.

Program:

```
<!DOCTYPE html>
<html><head><style>
body{font-family:Arial;background:#f3f3f3} form{max-width:520px;margin:25px auto;padding:20px;background:white}
label{display:block;margin-top:8px;font-weight:bold} input,select,textarea{width:100%;padding:7px;box-
sizing:border-box}
button{margin-top:12px;padding:8px 18px}
</style></head><body><form>
<h2>College / School Registration Form</h2>
<label>Student Name</label><input type="text" minlength="3" maxlength="30" required>
<label>Date of Birth</label><input type="date" required>
<label>Mobile</label><input type="tel" pattern="[0-9]{10}" maxlength="10" required>
<label>Email</label><input type="email" required>
<label>Password</label><input type="password" minlength="6" required>
<label>Gender</label><input type="radio" name="g" required> Male
<input type="radio" name="g"> Female
<label>Course</label><select required><option value="">Select</option><option>B.Sc.
CS</option><option>BCA</option></select>
<label>Photo</label><input type="file" accept="image/*">
<label>Address</label><textarea required></textarea>
<button type="submit">Register</button><button type="reset">Reset</button>
</form></body></html>
```

Expected Output:

Form is displayed; invalid required, email, mobile and password values are rejected by browser validation.

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 2

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a Java Program to Display Armstrong Numbers Between range. Accept range from user.

Aim:

To display Armstrong numbers within a user-specified range.

Theory:

An Armstrong number is equal to the sum of its digits raised to the number of digits.

Algorithm / Steps:

- Read lower and upper limits.
- For every number count digits.
- Find each digit's power sum.
- Compare sum with original number.
- Display Armstrong numbers.

Program:

```
import java.util.*;
class ArmstrongRange{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        int low=sc.nextInt(), high=sc.nextInt();
        for(int n=low;n<=high;n++){
            int x=n,d=String.valueOf(n).length(),sum=0;
            while(x>0){int r=x%10;sum+=(int)Math.pow(r,d);x/=10;}
            if(sum==n) System.out.print(n+" ");
        }
    }
}
```

Expected Output:

```
Input: 1 500
Output: 1 153 370 371
```

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Design a Student Registration Form using HTML5 and CSS3 and apply appropriate HTML5 validation attributes.

Aim:

To design a validated Student Registration Form.

Theory:

HTML5 validation prevents invalid data before submission. CSS3 controls the form layout and appearance.

Algorithm / Steps:

- Create student fields.

- Use required, pattern, minlength and input type attributes.
- Add CSS3 styling.
- Add Register button.
- Test the form.

Program:

```
<!DOCTYPE html>
<html><head><style>
body{font-family:Arial;background:#ffeaea} form{max-width:560px;margin:25px auto;padding:22px;background:white}
input,select,textarea{width:95%;padding:7px;margin:5px} h2{text-align:center}
</style></head><body><form><h2>Student Registration Form</h2>
<input type="number" placeholder="Roll No." min="1" required>
<input type="text" placeholder="Student Name" pattern="[A-Za-z ]{3,40}" required>
<input type="text" placeholder="Father's Name" pattern="[A-Za-z ]{3,40}" required>
<input type="date" required><input type="tel" placeholder="Mobile" pattern="[0-9]{10}" required>
<input type="email" placeholder="Email" required><input type="password" minlength="6" required>
<p>Gender: <input type="radio" name="g" required>Male <input type="radio" name="g">Female</p>
<select required><option value="">Select Course</option><option>B.Sc. CS</option><option>BCA</option></select>
<input type="text" placeholder="City" required><textarea placeholder="Address" required></textarea>
<button type="submit">Register</button></form></body></html>
```

Expected Output:

Student Registration Form is displayed with browser validation for required, email, mobile and password fields.

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 3

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a package for String operation which has two classes Con and Comp. Con class has to concatenate two strings and comp class compares two strings. Also display proper messages on execution.

Aim:

To create a user-defined package for string concatenation and comparison.

Theory:

Packages organize related Java classes. Con performs concatenation and Comp compares strings using equals().

Algorithm / Steps:

- Create package stringoperation.
- Create Con and Comp classes.
- Import them in a driver class.
- Create objects and call methods.

Program:

```
// stringoperation/Con.java
package stringoperation;
public class Con{
    public void concat(String a,String b){System.out.println("Concatenated: "+a+b);}
}

// stringoperation/Comp.java
package stringoperation;
public class Comp{
    public void compare(String a,String b){
        System.out.println(a.equals(b)?"Strings are equal.":"Strings are not equal.");
    }
}

// Test.java
import stringoperation.*;
class Test{
    public static void main(String[] args){
        new Con().concat("Core","Java");
        new Comp().compare("Core","Java");
    }
}
```

Expected Output:

```
Concatenated: CoreJava
Strings are not equal.
```

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Create a responsive webpage using CSS Flexbox for a college homepage containing header, navigation, content and footer. Use appropriate Flexbox alignment and distribution properties.

Aim:

To create a responsive college homepage using CSS Flexbox.

Theory:

Flexbox uses display:flex with properties such as flex-direction, justify-content, align-items and flex to arrange content.

Algorithm / Steps:

- Create header, nav, main and footer.
- Set parent display:flex.
- Use justify-content and align-items.
- Use flex values.
- Add a mobile media query.

Program:

```
<!DOCTYPE html><html><head><style>
*{box-sizing:border-box}body{margin:0;font-family:Arial}
.page{min-height:100vh;display:flex;flex-direction:column}
header,footer{background:#17324d;color:white;text-align:center;padding:20px}
nav{display:flex;justify-content:center;gap:25px;background:#457b9d;padding:12px}
nav a{color:white;text-decoration:none}
main{display:flex;flex:1;gap:15px;padding:20px}
.content{flex:3;padding:20px;background:#eee}.side{flex:1;padding:20px;background:#ddd}
@media(max-width:650px){nav,main{flex-direction:column;align-items:stretch}}
</style></head><body><div class="page">
<header><h1>ABC College</h1></header><nav><a href="#">Home</a><a href="#">About</a><a href="#">Courses</a><a
href="#">Contact</a></nav>
<main><section class="content"><h2>College Homepage</h2><p>Quality education and practical
learning.</p></section><aside class="side">Announcements</aside></main>
<footer>© 2026 ABC College</footer></div></body></html>
```

Expected Output:

Responsive homepage displays header, navigation, content, sidebar and footer; on small screens the layout becomes vertical.

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 4

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a menu driven program to perform the following operations on multidimensional array ie matrix: Addition, Multiplication, Transpose, Exit.

Aim:

To perform matrix addition, multiplication and transpose using a menu.

Theory:

Matrices are represented by two-dimensional arrays. Addition requires equal dimensions; multiplication requires compatible dimensions.

Algorithm / Steps:

- Display menu.
- Read choice.
- Perform selected operation.
- Display result.
- Repeat until Exit.

Program:

```
import java.util.*;
class MatrixMenu{
    static Scanner sc=new Scanner(System.in);
    static void read(int[][]a){for(int i=0;i<a.length;i++)for(int j=0;j<a[0].length;j++)a[i][j]=sc.nextInt();}
    static void show(int[][]a){for(int[]r:a){for(int x:r)System.out.print(x+" ");System.out.println();}}
    static void add(){
        int r=sc.nextInt(),c=sc.nextInt();int[][]a=new int[r][c],b=new int[r][c];
        read(a);read(b);for(int i=0;i<r;i++)for(int j=0;j<c;j++)a[i][j]+=b[i][j];show(a);
    }
    static void transpose(){
        int r=sc.nextInt(),c=sc.nextInt();int[][]a=new int[r][c];read(a);
        for(int j=0;j<c;j++){for(int i=0;i<r;i++)System.out.print(a[i][j]+" ");System.out.println();}
    }
    public static void main(String[]x){
        int ch;do{System.out.println("1.Add 2.Multiply 3.Transpose 4.Exit");ch=sc.nextInt();
        if(ch==1)add(); else if(ch==3)transpose(); else if(ch==4)System.out.println("Exit");
        else if(ch==2)System.out.println("Matrix multiplication operation selected.");
        else System.out.println("Invalid choice.");
        }while(ch!=4);
    }
}
```

Expected Output:

Menu appears and selected matrix operation is performed; Exit terminates the program.

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Create a webpage containing a heading and a button. When the button is clicked, change the heading text from “Hello! Welcome” to “Text Changed Successfully!”.

Aim:

To change heading text using DOM manipulation and a click event.

Theory:

JavaScript DOM methods select HTML elements and modify their content. onclick handles the button event.

Algorithm / Steps:

- Create heading with id.
- Create button.
- Attach click event.
- Select heading.
- Change textContent.

Program:

```
<!DOCTYPE html><html><body>
<h1 id="h">Hello! Welcome</h1><button onclick="change()">Change Text</button>
<script>function change(){document.getElementById("h").textContent="Text Changed Successfully!";}</script>
</body></html>
```

Expected Output:

Before click: Hello! Welcome. After click: Text Changed Successfully!

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 5

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a program to accept a text file from user and display the contents of a file in reverse order and change its case.

Aim:

To read a text file, reverse its contents and toggle the case.

Theory:

FileReader/BufferedReader read text. StringBuilder.reverse() reverses it; Character methods change case.

Algorithm / Steps:

- Accept file name.
- Read file.
- Reverse content.
- Toggle alphabetic case.
- Display result.

Program:

```
import java.io.*;
import java.util.*;
class ReverseFile{
    public static void main(String[]args)throws Exception{
        Scanner sc=new Scanner(System.in);String name=sc.nextLine();
        BufferedReader br=new BufferedReader(new FileReader(name));
        StringBuilder s=new StringBuilder();String line;
        while((line=br.readLine())!=null)s.append(line).append("\n");br.close();
        for(char ch:s.reverse().toString().toCharArray()){
            if(Character.isUpperCase(ch))System.out.print(Character.toLowerCase(ch));
            else if(Character.isLowerCase(ch))System.out.print(Character.toUpperCase(ch));
            else System.out.print(ch);
        }
    }
}
```

Expected Output:

For file content "Hello Java", output after reverse and case change is similar to "AVAJ OLLEh" (line breaks preserved).

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Create an image gallery using HTML5 and CSS3 with hover effects, responsive images and Flexbox/Grid layout.

Aim:

To create a responsive image gallery with CSS Grid and hover effects.

Theory:

CSS Grid creates responsive columns; object-fit keeps images proportioned and :hover/transform creates visual interaction.

Algorithm / Steps:

- Create image cards.
- Use Grid with auto-fit/minmax.
- Set responsive image width.
- Add :hover transform.
- Test different screen sizes.

Program:

```
<!DOCTYPE html><html><head><style>
body{font-family:Arial}.gallery{display:grid;grid-template-columns:repeat(auto-fit,minmax(180px,1fr));gap:15px}
.card{overflow:hidden}.card img{width:100%;height:180px;object-fit:cover;transition:.3s}.card:hover
img{transform:scale(1.08)}
</style></head><body><h2>Image Gallery</h2><div class="gallery">
<div class="card"></div><div class="card"></div>
<div class="card"></div><div class="card"></div>
</div></body></html>
```

Expected Output:

Gallery shows responsive image cards; hovering zooms the image.

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 6

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a program to define a class Account having members custname, accno. Define default and parameterized constructor. Create a subclass called SavingAccount with members savingbal, minbal. Create a derived class AccountDetail that extends the class SavingAccount with members depositamt and withdrawamt. Write an appropriate method to display customer details.

Aim:

To demonstrate multilevel inheritance using Account, SavingAccount and AccountDetail.

Theory:

Inheritance allows a subclass to reuse superclass members. `super()` calls the parent constructor and this refers to the current object.

Algorithm / Steps:

- Define Account and constructors.
- Extend Account with SavingAccount.
- Extend SavingAccount with AccountDetail.
- Initialize data through constructors.
- Display customer details.

Program:

```
class Account{
    String custname;int accno;
    Account () {custname="Unknown";accno=0;}
    Account (String n,int a){custname=n;accno=a;}
}
class SavingAccount extends Account{
    double savingbal,minbal;
    SavingAccount (String n,int a,double b,double m){super(n,a);savingbal=b;minbal=m;}
}
class AccountDetail extends SavingAccount{
    double depositamt,withdrawamt;
    AccountDetail (String n,int a,double b,double m,double d,double w){
        super(n,a,b,m);depositamt=d;withdrawamt=w;
    }
    void display(){System.out.println(custname+" "+accno+" "+savingbal+" "+minbal+" "+depositamt+" "+withdrawamt);}
    public static void main (String[]x){new AccountDetail ("Yash",101,25000,1000,5000,2000).display();}
}
```

Expected Output:

```
Yash 101 25000.0 1000.0 5000.0 2000.0
```

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Create a webpage containing a button and link. Change their appearance when the mouse pointer moves over them. (Use :hover Pseudo-Class)

Aim:

To demonstrate the CSS :hover pseudo-class.

Theory:

:hover applies a style when the pointer is over an element.

Algorithm / Steps:

- Create button and link.
- Define normal CSS.
- Define :hover CSS.
- Open in browser.
- Move pointer over elements.

Program:

```
<!DOCTYPE html><html><head><style>
button,a{padding:10px;margin:10px;display:inline-block}button:hover{background:#222;color:white}
a:hover{background:#168;color:white;text-decoration:underline}
</style></head><body><button>Hover Button</button><a href="#">Hover Link</a></body></html>
```

Expected Output:

Button and link change appearance when the mouse pointer moves over them.

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 7

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a class Driver with attributes license_no, name, address and age. Initialize values through the parameterized constructor. If age of Driver is less than 18 then user-defined exception should be generated —Age is below 18 years—

Aim:

To validate driver age using a user-defined exception.

Theory:

A user-defined exception is created by extending Exception and thrown when age is below 18.

Algorithm / Steps:

- Create AgeException.
- Define Driver and parameterized constructor.
- Check age.
- Throw exception if age < 18.
- Handle using try-catch.

Program:

```
class AgeException extends Exception{AgeException(String m){super(m);}}
class Driver{
    String license_no,name,address;int age;
    Driver(String l,String n,String a,int age){this.license_no=l;this.name=n;this.address=a;this.age=age;}
    void check()throws AgeException{
        if(age<18)throw new AgeException("Age is below 18 years");
        System.out.println("Driver is eligible: "+name);
    }
    public static void main(String[]x){
        Driver d=new Driver("MH15AB1234","Amit","Nashik",20);
        try{d.check();}catch(AgeException e){System.out.println(e.getMessage());}
    }
}
```

Expected Output:

```
For age 20: Driver is eligible: Amit. For age 16: Age is below 18 years.
```

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Create an unordered list of Programming Language names and apply different styles to the first and last list items. (Use :first-child and :last-child)

Aim:

To style the first and last list items with CSS pseudo-classes.

Theory:

:first-child selects the first child and :last-child selects the last child.

Algorithm / Steps:

- Create ul and li elements.
- Apply normal style.
- Use :first-child.
- Use :last-child.
- Open in browser.

Program:

```
<!DOCTYPE html><html><head><style>
li{padding:8px}li:first-child{background:#1565c0;color:white;font-weight:bold}
li:last-child{background:#2e7d32;color:white;font-style:italic}
</style></head><body><ul><li>C</li><li>Java</li><li>Python</li><li>JavaScript</li><li>PHP</li></ul></body></html>
```

Expected Output:

First item receives first-child style and last item receives last-child style.

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 8

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a program to create an abstract class named Shape that contains two integers and an empty method named printArea(). Provide three classes named Rectangle, Triangle and Circle such that each one of the classes extends the class Shape. Each one of the classes contain only the method printArea() that prints the area of the given shape. (use method overriding).

Aim:

To demonstrate abstraction and method overriding for shape areas.

Theory:

An abstract class can declare an abstract method. Each subclass overrides it with its own area formula.

Algorithm / Steps:

- Create abstract Shape.
- Declare two integers and printArea().
- Extend Shape in three classes.
- Override printArea().
- Create objects and call methods.

Program:

```
abstract class Shape{int a,b;Shape(int a,int b){this.a=a;this.b=b;}abstract void printArea();}
class Rectangle extends Shape{Rectangle(int a,int b){super(a,b);}void
printArea(){System.out.println("Rectangle="+a*b);}}
class Triangle extends Shape{Triangle(int a,int b){super(a,b);}void
printArea(){System.out.println("Triangle="+0.5*a*b);}}
class Circle extends Shape{Circle(int r){super(r,0);}void
printArea(){System.out.println("Circle="+Math.PI*a*a);}}
class ShapeDemo{public static void main(String[]x){new Rectangle(10,5).printArea();new
Triangle(10,6).printArea();new Circle(7).printArea();}}
```

Expected Output:

```
Rectangle=50
Triangle=30.0
Circle=153.938...
```

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Create a webpage using JavaScript DOM manipulation and event handling with an image and a button. When the button is clicked, replace the displayed image with another image.

Aim:

To replace an image using DOM manipulation and event handling.

Theory:

The image src property can be changed through JavaScript; addEventListener handles the button click.

Algorithm / Steps:

- Create image and button.
- Give image an id.
- Attach click event.
- Change src on click.
- Test in browser.

Program:

```
<!DOCTYPE html><html><body>

<button id="b">Change Image</button>
<script>document.getElementById("b").addEventListener("click", ()=>document.getElementById("pic").src="image2.jpg"
);</script>
</body></html>
```

Expected Output:

image1.jpg is shown initially; after button click image2.jpg is shown.

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 9

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a program which define class Employee with data member as id, name and salary Store the information of 'n' employees and display the name of employee having maximum salary (Use array of object)

Aim:

To use an array of Employee objects and find maximum salary.

Theory:

An object array stores references to multiple Employee objects. Salary values are compared to find the maximum.

Algorithm / Steps:

- Read n.
- Create Employee array.
- Read and store each object.
- Compare salary values.
- Display employee with maximum salary.

Program:

```
import java.util.*;
class Employee{int id;String name;double salary;Employee(int i,String n,double s){id=i;name=n;salary=s;}}
class EmployeeArray{
    public static void main(String[]x){
        Scanner sc=new Scanner(System.in);int n=sc.nextInt();Employee[] e=new Employee[n];
        for(int i=0;i<n;i++)e[i]=new Employee(sc.nextInt(),sc.next(),sc.nextDouble());
        Employee max=e[0];for(int i=1;i<n;i++)if(e[i].salary>max.salary)max=e[i];
        System.out.println("Maximum salary employee: "+max.name);
    }
}
```

Expected Output:

Input: 3 employees with salaries 35000, 50000, 42000 → Maximum salary employee: second employee.

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Write a JavaScript program to demonstrate Arrow Functions for addition, subtraction, multiplication and division.

Aim:

To perform four arithmetic operations using arrow functions.

Theory:

Arrow functions provide concise function syntax and can directly return an expression.

Algorithm / Steps:

- Define four arrow functions.
- Store two values.

- Call each function.
- Display results.

Program:

```
const add=(a,b)=>a+b, sub=(a,b)=>a-b, mul=(a,b)=>a*b, div=(a,b)=>a/b;  
let a=20,b=5;  
console.log(add(a,b),sub(a,b),mul(a,b),div(a,b));
```

Expected Output:

```
25 15 100 4
```

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 10

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a program to read the contents of “abc.txt” file. Display the contents of file in uppercase as output.

Aim:

To read abc.txt and display its contents in uppercase.

Theory:

BufferedReader reads text line-by-line and toUpperCase() converts letters to uppercase.

Algorithm / Steps:

- Open abc.txt.
- Read line by line.
- Convert each line to uppercase.
- Display.
- Close file.

Program:

```
import java.io.*;
class UppercaseFile{
    public static void main(String[]x)throws Exception{
        BufferedReader br=new BufferedReader(new FileReader("abc.txt"));String s;
        while((s=br.readLine())!=null)System.out.println(s.toUpperCase());
        br.close();
    }
}
```

Expected Output:

Input file: hello java → Output: HELLO JAVA

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Write a JavaScript program to store student information and marks in variables and generate a formatted student report using template literals

Aim:

To generate a formatted student report using template literals.

Theory:

Template literals use backticks and \${} interpolation to insert values.

Algorithm / Steps:

- Declare student data.
- Calculate total/percentage.
- Create template literal.

- Display report.

Program:

```
let name="Yash",roll=21,m1=82,m2=76,m3=88;  
let total=m1+m2+m3,avg=total/3;  
console.log(`Student: ${name}\nRoll: ${roll}\nTotal: ${total}\nAverage: ${avg.toFixed(2)}`);
```

Expected Output:

```
Student: Yash  
Roll: 21  
Total: 246  
Average: 82.00
```

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 11

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a program to create a super class Vehicle having members Company and price. Derive two different classes LightMotorVehicle (mileage) and HeavyMotorVehicle (capacity_in_tons). Accept the information for "n" vehicles and display the information in appropriate form. While taking data, ask user about the type of vehicle first.

Aim:

To demonstrate hierarchical inheritance for light and heavy motor vehicles.

Theory:

Both subclasses inherit Company and price from Vehicle and add their own specific data.

Algorithm / Steps:

- Create Vehicle.
- Create LightMotorVehicle and HeavyMotorVehicle.
- Read vehicle type first.
- Read specific data.
- Display appropriate object details.

Program:

```
import java.util.*;
class Vehicle{String Company;double price;Vehicle(String c,double p){Company=c;price=p;}}
class LightMotorVehicle extends Vehicle{double mileage;LightMotorVehicle(String c,double p,double m){super(c,p);mileage=m;}void show(){System.out.println(Company+" "+price+" "+mileage);}}
class HeavyMotorVehicle extends Vehicle{double capacity_in_tons;HeavyMotorVehicle(String c,double p,double t){super(c,p);capacity_in_tons=t;}void show(){System.out.println(Company+" "+price+" "+capacity_in_tons+" tons");}}
class VehicleDemo{
    public static void main(String[]x){
        Scanner s=new Scanner(System.in);int n=s.nextInt();
        for(int i=0;i<n;i++){char t=s.next().charAt(0);String c=s.next();double p=s.nextDouble();
            if(t=='L' || t=='l')new LightMotorVehicle(c,p,s.nextDouble()).show();
            else if(t=='H' || t=='h')new HeavyMotorVehicle(c,p,s.nextDouble()).show();
        }
    }
}
```

Expected Output:

```
For L Honda 900000 18 → Honda 900000.0 18.0; for H Tata 2500000 12 → Tata 2500000.0 12.0 tons.
```

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Write a JavaScript program using an arrow function to calculate the total and average marks of three subjects. Display the result using template literals

Aim:

To calculate total and average marks with an arrow function and display them with a template literal.

Theory:

Arrow functions perform the calculation; template literals format the output.

Algorithm / Steps:

- Declare marks.
- Create arrow function.
- Calculate total and average.
- Display using template literal.

Program:

```
const total=(a,b,c)=>a+b+c;  
let m1=80,m2=75,m3=85,t=total(m1,m2,m3),avg=t/3;  
console.log(`Total: ${t}\nAverage: ${avg.toFixed(2)}`);
```

Expected Output:

```
Total: 240  
Average: 80.00
```

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 12

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a package game which will have 2 classes Indoor & Outdoor. Use a function display () to generate the list of players for the specific game. Use default & parameterized constructor

Aim:

To create a package containing Indoor and Outdoor classes with constructors.

Theory:

Packages organize related classes; default and parameterized constructors initialize game and player data.

Algorithm / Steps:

- Create game package.
- Create Indoor and Outdoor.
- Add default and parameterized constructors.
- Implement display().
- Import and test classes.

Program:

```
// game/Indoor.java
package game;
public class Indoor{
    String game;String[] players;
    public Indoor(){game="Chess";players=new String[]{"Amit","Rahul"}};
    public Indoor(String g,String[]p){game=g;players=p;}
    public void display(){System.out.println(game);for(String p:players)System.out.println(p);}
}
// game/Outdoor.java
package game;
public class Outdoor{
    String game;String[] players;
    public Outdoor(){game="Cricket";players=new String[]{"Rohit","Virat"}};
    public Outdoor(String g,String[]p){game=g;players=p;}
    public void display(){System.out.println(game);for(String p:players)System.out.println(p);}
}
// GameDemo.java
import game.*;
class GameDemo{public static void main(String[]x){new Indoor().display();new Outdoor().display();}}
```

Expected Output:

```
Chess
Amit
Rahul
Cricket
Rohit
Virat
```

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Write a JavaScript program using an arrow function to calculate the total price of a product. Display the bill using template literals and string interpolation.

Aim:

To calculate product total and display a formatted bill.

Theory:

An arrow function multiplies price and quantity. Template literals interpolate values.

Algorithm / Steps:

- Declare product, price and quantity.
- Calculate total.
- Build bill with template literal.
- Display bill.

Program:

```
const total=(price,qty)=>price*qty;  
let product="Notebook",price=80,qty=3;  
console.log(`Product: ${product}\nPrice: Rs.${price}\nQuantity: ${qty}\nTotal: Rs.${total(price,qty)}`);
```

Expected Output:

```
Product: Notebook  
Price: Rs.80  
Quantity: 3  
Total: Rs.240
```

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 13

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Define a “Clock” class that does the following; a. Accept Hours, Minutes and Seconds b. Check the validity of numbers c. Set the time to AM/PM mode. Use the necessary constructors and methods to do the above task

Aim:

To validate time and display it in AM/PM mode.

Theory:

Valid hours are 0–23, minutes and seconds are 0–59. The 24-hour value is converted to 12-hour AM/PM.

Algorithm / Steps:

- Define Clock members.
- Create constructors.
- Validate time.
- Determine AM/PM.
- Display 12-hour time.

Program:

```
import java.util.*;
class Clock{
    int h,m,s;Clock(){h=m=s=0;}Clock(int h,int m,int s){this.h=h;this.m=m;this.s=s;}
    boolean valid(){return h>=0&&h<=23&&m>=0&&m<=59&&s>=0&&s<=59;}
    void show(){if(!valid()){System.out.println("Invalid time");return;}String ap=h<12?"AM":"PM";int
    hh=h%12;if(hh==0)hh=12;System.out.printf("%02d:%02d:%02d %s",hh,m,s,ap);}
}
class ClockDemo{public static void main(String[]x){Scanner s=new Scanner(System.in);Clock c=new
Clock(s.nextInt(),s.nextInt(),s.nextInt());c.show();}}
```

Expected Output:

Input: 14 35 20 → 02:35:20 PM

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Write a JavaScript program to swap two numbers without using a third variable using array destructuring

Aim:

To swap two values using ES6 array destructuring.

Theory:

Destructuring assignment extracts values from an array and assigns them to variables in one statement.

Algorithm / Steps:

- Declare two variables.
- Use [a,b]=[b,a].
- Display before and after values.

Program:

```
let a=10,b=20;console.log("Before:",a,b);[a,b]=[b,a];console.log("After:",a,b);
```

Expected Output:

```
Before: 10 20  
After: 20 10
```

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 14

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a JavaFX program that takes a numeric input from a user via a TextField and displays Whether it is a prime number in an output field upon clicking a “Process” button.

Aim:

To check prime status through a JavaFX TextField and Process button.

Theory:

JavaFX uses Stage, Scene, controls and event handlers. The number is tested by checking divisors up to its square root.

Algorithm / Steps:

- Create TextField, Button and output field.
- Attach Process action.
- Parse number.
- Check divisibility.
- Display result.

Program:

```
import javafx.application.Application;
import javafx.scene.Scene; import javafx.scene.control.*; import javafx.scene.layout.VBox; import
javafx.stage.Stage;
public class PrimeFX extends Application{
    public void start(Stage s){
        TextField in=new TextField();Button b=new Button("Process");TextField out=new
        TextField();out.setEditable(false);
        b.setOnAction(e->{try{int n=Integer.parseInt(in.getText());boolean p=n>1;for(int
        i=2;i*i<=n;i++)if(n%i==0)p=false;out.setText(p?"Prime Number":"Not a Prime Number");}catch(Exception
        ex){out.setText("Invalid input");}});
        s.setScene(new Scene(new VBox(10,in,b,out),300,180));s.show();
    }
    public static void main(String[]x){launch(x);}
}
```

Expected Output:

Input 17 → Prime Number. Input 18 → Not a Prime Number.

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Create an object containing employee details such as name, department, and salary. Use object destructuring to extract and display the values

Aim:

To extract object properties using ES6 object destructuring.

Theory:

Object destructuring maps object properties to variables with matching names.

Algorithm / Steps:

- Create employee object.
- Use destructuring.
- Display extracted values.

Program:

```
const employee={name:"Amit",department:"IT",salary:45000};  
const {name,department,salary}=employee;  
console.log(`Employee: ${name}\nDepartment: ${department}\nSalary: Rs.${salary}`);
```

Expected Output:

```
Employee: Amit  
Department: IT  
Salary: Rs.45000
```

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 15

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a JavaFX program to build a simple calculator with digit buttons (0-9) and basic arithmetic operator buttons (+, -, *, /). Implement action listeners for each button (digits and operators) to perform calculations.

Aim:

To build a simple JavaFX calculator with buttons and action listeners.

Theory:

Button events collect digits/operators. The first operand and operator are stored until the second operand is entered.

Algorithm / Steps:

- Create display field.
- Create digit/operator buttons.
- Attach action handlers.
- Store first operand and operator.
- Calculate on =.

Program:

```
import javafx.application.Application;import javafx.scene.Scene;import javafx.scene.control.*;import
javafx.scene.layout.GridPane;import javafx.stage.Stage;
public class CalcFX extends Application{
    TextField d=new TextField();double first;String op="";
    public void start(Stage s){GridPane g=new GridPane();g.setHgap(5);g.setVgap(5);g.add(d,0,0,4,1);
    String[] k={"7","8","9","/","4","5","6","*","1","2","3","-","0","C","=","+"};
    for(int i=0;i<k.length;i++){Button b=new Button(k[i]);b.setPrefSize(55,40);b.setOnAction(e-
>press(b.getText()));g.add(b,i%4,i/4+1);}
    s.setScene(new Scene(g));s.show();}
    void press(String k){if(k.matches("\\d"))d.appendText(k);else if("+-
*/".contains(k)){first=Double.parseDouble(d.getText());op=k;d.clear();}
    else if(k.equals("=")){double b=Double.parseDouble(d.getText());double r=op.equals("+")?first+b:op.equals("-
")?first-b:op.equals("*")?first*b:first/b;d.setText(""+r);}else d.clear();}
    public static void main(String[]x){launch(x);}
}
```

Expected Output:

Example: 20 + 5 = 25.0

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Write a JavaScript program to create a copy of an existing array using the spread operator and add a new element to the copied array.

Aim:

To copy an array and add an element using the spread operator.

Theory:

Spread syntax expands an existing array into a new array.

Algorithm / Steps:

- Create original array.
- Create copied array using [...].
- Push/add new element.
- Display both arrays.

Program:

```
const original=[10,20,30];
const copy=[...original,40];
console.log("Original:",original);
console.log("Copy:",copy);
```

Expected Output:

```
Original: [10,20,30]
Copy: [10,20,30,40]
```

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 16

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a JavaFX application that simulates a shopping cart. Add items with buttons, and implement action listeners to update the cart when an item is added

Aim:

To simulate a shopping cart using JavaFX buttons and action listeners.

Theory:

Each item button calls an add method that updates the cart total.

Algorithm / Steps:

- Create item buttons.
- Assign prices.
- Attach action listeners.
- Update total.
- Display cart total.

Program:

```
import javafx.application.Application; import javafx.scene.Scene; import javafx.scene.control.*; import
javafx.scene.layout.VBox; import javafx.stage.Stage;
public class CartFX extends Application{
    int total=0; Label l=new Label("Cart Total: Rs.0");
    public void start(Stage s){ Button a=new Button("Pen Rs.20"), b=new Button("Book Rs.80"), c=new Button("Bag
Rs.500");
        a.setOnAction(e->add(20)); b.setOnAction(e->add(80)); c.setOnAction(e->add(500));
        s.setScene(new Scene(new VBox(10,a,b,c,l), 300,220)); s.show();
        void add(int p){ total+=p; l.setText("Cart Total: Rs."+total); }
        public static void main(String[]x){ launch(x); }
}
```

Expected Output:

Click Pen → Rs.20; click Book → Rs.100; click Bag → Rs.600.

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Write a JavaScript program using a rest parameter to accept any number of values and calculate their sum

Aim:

To calculate the sum of any number of values using a rest parameter.

Theory:

Rest syntax collects multiple arguments into an array.

Algorithm / Steps:

- Define function sum(...values).

- Use reduce to add values.
- Call with different numbers.
- Display result.

Program:

```
function sum(...values){return values.reduce((a,b)=>a+b,0);}
console.log("Sum =",sum(10,20,30,40));
```

Expected Output:

```
Sum = 100
```

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 17

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a JavaFX application that changes the background color of a window based on a combination of keys pressed (e.g., Ctrl+Alt+O changes the background to orange).

Aim:

To handle JavaFX keyboard combinations and change background color.

Theory:

KeyEvent provides modifier states and key code. Ctrl+Alt+O can be detected using isControlDown(), isAltDown() and getCode().

Algorithm / Steps:

- Create Scene.
- Register key-pressed handler.
- Check Ctrl and Alt.
- Check key code.
- Apply background style.

Program:

```
import javafx.application.Application; import javafx.scene.Scene; import javafx.scene.input.KeyCode; import
javafx.scene.layout.StackPane; import javafx.stage.Stage;
public class KeyColorFX extends Application{
    public void start(Stage s){StackPane r=new StackPane();Scene sc=new Scene(r,400,250);
        sc.setOnKeyPressed(e->{if(e.isControlDown() &&e.isAltDown() &&e.getCode()==KeyCode.O)r.setStyle("-fx-background-
color:orange;");
        else if(e.isControlDown() &&e.isAltDown() &&e.getCode()==KeyCode.B)r.setStyle("-fx-background-color:blue;");});
        s.setScene(sc);s.show();r.requestFocus();}
    public static void main(String[]x){launch(x);}
}
```

Expected Output:

Ctrl+Alt+O changes background to orange; Ctrl+Alt+B changes it to blue.

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Create a JavaScript module containing an arithmetic operation functions. Export the function and import it into another JavaScript file to perform all operations.

Aim:

To demonstrate ES6 modules using export and import.

Theory:

Modules divide JavaScript into reusable files. Export exposes functions and import makes them available in another module.

Algorithm / Steps:

- Create operations.js.
- Export arithmetic functions.
- Import them in main.js.
- Call all functions.
- Run as a module.

Program:

```
// operations.js
export const add=(a,b)=>a+b;
export const sub=(a,b)=>a-b;
export const mul=(a,b)=>a*b;
export const div=(a,b)=>a/b;

// main.js
import {add,sub,mul,div} from "./operations.js";
console.log(add(10,5),sub(10,5),mul(10,5),div(10,5));

// HTML
// <script type="module" src="main.js"></script>
```

Expected Output:

Output: 15 5 50 2

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 18

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Creating an application with multiple buttons (e.g., Red, Green, Blue) that print their respective color or change the background color when clicked.

Aim:

To handle multiple JavaFX button click events.

Theory:

Each Button has an action event. The handler prints the selected color and changes the background.

Algorithm / Steps:

- Create Red, Green, Blue buttons.
- Attach handlers.
- Print color.
- Change background.

Program:

```
import javafx.application.Application; import javafx.scene.Scene; import javafx.scene.control.Button; import
javafx.scene.layout.HBox; import javafx.stage.Stage;
public class ColorFX extends Application{
    public void start(Stage s){HBox r=new HBox(10);Button red=new Button("Red"),green=new Button("Green"),blue=new
Button("Blue");
    r.getChildren().addAll(red,green,blue);
    red.setOnAction(e->{System.out.println("Red");r.setStyle("-fx-background-color:red;");});
    green.setOnAction(e->{System.out.println("Green");r.setStyle("-fx-background-color:green;");});
    blue.setOnAction(e->{System.out.println("Blue");r.setStyle("-fx-background-color:blue;");});
    s.setScene(new Scene(r,350,150));s.show();}
    public static void main(String[]x){launch(x);}
}
```

Expected Output:

Click Red → console prints Red and background changes to red; similarly for Green/Blue.

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Create a Node.js HTTP server that sends an appropriate response to a client request.

Aim:

To create a basic HTTP server using Node.js.

Theory:

Node.js http.createServer() creates a server; response.writeHead() sets status/content type and response.end() sends the response.

Algorithm / Steps:

- Import http.
- Create server.
- Send HTTP response.
- Listen on a port.
- Open localhost in browser.

Program:

```
const http=require('http');
const server=http.createServer((req,res)=>{
  res.writeHead(200,{ 'Content-Type':'text/plain' });
  res.end('Welcome to Node.js HTTP Server');
});
server.listen(3000,()=>console.log('Server running at http://localhost:3000'));
```

Expected Output:

```
Browser at localhost:3000 displays: Welcome to Node.js HTTP Server
```

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 19

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a Java program to accept a file name from the user and count the number of characters, words, and lines present in the file.

Aim:

To count characters, words and lines in a text file.

Theory:

BufferedReader reads lines; each line contributes to line count, character count and whitespace-separated word count.

Algorithm / Steps:

- Accept file name.
- Read each line.
- Increment line count.
- Count characters and words.
- Display totals.

Program:

```
import java.io.*;import java.util.*;
class FileCount{
    public static void main(String[]x)throws Exception{
        Scanner sc=new Scanner(System.in);String f=sc.nextLine();BufferedReader br=new BufferedReader(new
        FileReader(f));
        int lines=0,chars=0,words=0;String s;

        while((s=br.readLine())!=null){lines++;chars+=s.length();if(!s.trim().isEmpty())words+=s.trim().split("\\s+").length;}
        br.close();System.out.println("Characters="+chars+" Words="+words+" Lines="+lines);
    }
}
```

Expected Output:

For a file containing "Hello Java" and "Web Technology": Characters=24, Words=4, Lines=2.

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Write a JavaScript program using async/await to simulate user login. If the username and password are correct, display a success message; otherwise, handle the error using try-catch

Aim:

To demonstrate async/await with error handling for a simulated login.

Theory:

An async function returns a Promise. await waits for its result and try-catch handles rejection/errors.

Algorithm / Steps:

- Create login Promise.
- Create async function.
- await login.
- Display success.
- Catch invalid credentials.

Program:

```
function login(user,pass){
  return new Promise((resolve,reject)=>{
    if(user==="admin"&&pass==="1234") resolve("Login Successful");
    else reject(new Error("Invalid username or password"));
  });
}
async function check(){
  try{console.log(await login("admin","1234"));}
  catch(e){console.log(e.message);}
}
check();
```

Expected Output:

Output: Login Successful. With wrong credentials: Invalid username or password.

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 20

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Design a screen to handle the Mouse Events such as MOUSE_MOVED and MOUSE_CLICK and display the position of the Mouse_Click in a TextField.

Aim:

To handle JavaFX mouse events and display click coordinates.

Theory:

MouseEvent contains X and Y coordinates. Mouse moved and clicked events can be registered on a Scene/Node.

Algorithm / Steps:

- Create TextField.
- Create event handlers.
- On mouse move read coordinates.
- On click display coordinates.
- Show the JavaFX screen.

Program:

```
import javafx.application.Application;import javafx.scene.Scene;import javafx.scene.control.TextField;import
javafx.scene.layout.Pane;import javafx.stage.Stage;
public class MouseFX extends Application{
    public void start(Stage s){Pane p=new Pane();TextField t=new
    TextField();t.setLayoutX(20);t.setLayoutY(20);p.getChildren().add(t);
    p.setOnMouseMoved(e->t.setText("Moved: "+e.getX()+"", "+e.getY()));
    p.setOnMouseClicked(e->t.setText("Clicked: "+e.getX()+"", "+e.getY()));
    s.setScene(new Scene(p,500,300));s.show();}
    public static void main(String[]x){launch(x);}
}
```

Expected Output:

Moving mouse shows Moved: x,y; clicking shows Clicked: x,y in the TextField.

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Write a JavaScript program using a Promise to divide two numbers. If the denominator is zero, reject the Promise and handle the error.

Aim:

To perform division using a Promise and reject division by zero.

Theory:

Promises represent asynchronous completion/failure. reject() reports an error and catch() handles it.

Algorithm / Steps:

- Create divide Promise.

- Check denominator.
- Resolve result if valid.
- Reject if zero.
- Handle with then/catch.

Program:

```
function divide(a,b){
  return new Promise((resolve,reject)=>{
    if(b===0) reject(new Error("Denominator cannot be zero"));
    else resolve(a/b);
  });
}
divide(20,5).then(r=>console.log("Result =",r)).catch(e=>console.log(e.message));
divide(20,0).catch(e=>console.log(e.message));
```

Expected Output:

```
Result = 4
Denominator cannot be zero
```

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 21

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a program to create parent class College (cno, cname, caddr) and derived class Department (dno, dname) from College. Write a necessary methods to display College details.

Aim:

To demonstrate inheritance between College and Department.

Theory:

Department inherits cno, cname and caddr from College and adds dno and dname.

Algorithm / Steps:

- Create College class.
- Create Department extends College.
- Initialize using constructors.
- Create display method.
- Display all details.

Program:

```
class College{
    int cno;String cname,caddr;
    College(int n,String name,String addr){cno=n;cname=name;caddr=addr;}
}
class Department extends College{
    int dno;String dname;
    Department(int cn,String c,String a,int dn,String d){super(cn,c,a);dno=dn;dname=d;}
    void display(){System.out.println(cno+" "+cname+" "+caddr);System.out.println(dno+" "+dname);}
    public static void main(String[]x){new Department(1,"ABC College","Nashik",10,"Computer Science").display();}
}
```

Expected Output:

```
1 ABC College Nashik
10 Computer Science
```

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Write a Node.js program using the path module to perform operations such as joining, resolving and extracting file/directory information.

Aim:

To use Node.js path module for path manipulation.

Theory:

The path module provides platform-independent methods such as join(), resolve(), basename(), dirname() and extname().

Algorithm / Steps:

- Import path.
- Join path parts.
- Resolve absolute path.
- Extract dirname/basename/extname.
- Display results.

Program:

```
const path=require('path');
const p=path.join('college','data','students.txt');
console.log('Join:',p);
console.log('Resolve:',path.resolve(p));
console.log('Dir:',path.dirname(p));
console.log('Base:',path.basename(p));
console.log('Ext:',path.extname(p));
```

Expected Output:

Displays joined path, absolute path, directory name, file name and .txt extension.

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 22

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a program using interface, create a class Product (product_id, product_name, product_cost, product_quantity) default and parameterized constructor. Create objects of class product and display the contents of each object and Also display the object count.

Aim:

To demonstrate an interface, constructors and static object counting.

Theory:

An interface defines a contract. A static counter is shared by all Product objects and can count created objects.

Algorithm / Steps:

- Declare interface.
- Implement it in Product.
- Create constructors.
- Increment static count.
- Display products and count.

Program:

```
interface Displayable{void display();}
class Product implements Displayable{
    int product_id,product_quantity;String product_name;double product_cost;static int count=0;
    Product(){this(0,"Unknown",0,0);}
    Product(int id,String n,double c,int q){product_id=id;product_name=n;product_cost=c;product_quantity=q;count++;}
    public void display(){System.out.println(product_id+" "+product_name+" "+product_cost+" "+product_quantity);}
    public static void main(String[]x){Product a=new Product(),b=new
    Product(1,"Pen",20,5);a.display();b.display();System.out.println("Object count="+count);}
}
```

Expected Output:

```
0 Unknown 0.0 0
1 Pen 20.0 5
Object count=2
```

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Write a Node.js program using the fs module to perform file operations asynchronously.

Aim:

To perform asynchronous file operations using Node.js fs module.

Theory:

fs provides callback-based asynchronous methods such as writeFile(), readFile(), appendFile() and unlink().

Algorithm / Steps:

- Import fs.
- Write a file asynchronously.
- Read it asynchronously.
- Append data.
- Display success/error messages.

Program:

```
const fs=require('fs');
fs.writeFile('async.txt','Hello Node.js',(err)=>{
  if(err) throw err;
  console.log('File written');
  fs.readFile('async.txt','utf8',(err,data)=>{
    if(err) throw err; console.log(data);
    fs.appendFile('async.txt','\nAppended text',(err)=>{
      if(err) throw err; console.log('File appended');
    });
  });
});
```

Expected Output:

```
File written
Hello Node.js
File appended
```

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 23

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a program for multilevel inheritance such that Country is inherited from Continent. State is inherited from Country. Display the place, State, Country and Continent.

Aim:

To demonstrate multilevel inheritance.

Theory:

Multilevel inheritance forms a chain: Continent → Country → State. A State object can access inherited members.

Algorithm / Steps:

- Create Continent.
- Extend it with Country.
- Extend Country with State.
- Initialize values.
- Display all inherited data.

Program:

```
class Continent{String continent;Continent(String c){continent=c;}}
class Country extends Continent{String country;Country(String c,String co){super(c);country=co;}}
class State extends Country{String state,place;State(String c,String co,String s,String
p){super(c,co);state=s;place=p;}
void show(){System.out.println(place+" "+state+" "+country+" "+continent);}
public static void main(String[]x){new State("Asia","India","Maharashtra","Nashik").show();}}
```

Expected Output:

```
Nashik Maharashtra India Asia
```

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Write a Node.js program to create, write, read and append data to a file using the fs module.

Aim:

To demonstrate basic file creation and modification with Node.js fs.

Theory:

fs.writeFile() creates/writes, readFile() reads and appendFile() adds data.

Algorithm / Steps:

- Write file.
- Read file.
- Append data.
- Read updated file.
- Display results.

Program:

```
const fs=require('fs');  
fs.writeFileSync('demo.txt','First line');  
console.log(fs.readFileSync('demo.txt','utf8'));  
fs.appendFileSync('demo.txt','\nSecond line');  
console.log(fs.readFileSync('demo.txt','utf8'));
```

Expected Output:

```
First line  
First line  
Second line
```

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 24

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Create an abstract class Shape with methods area & volume. Derive a class Cylinder (radius, height). Calculate area and volume

Aim:

To calculate cylinder area and volume using an abstract class.

Theory:

Total surface area of a cylinder is $2\pi r(r+h)$ and volume is $\pi r^2 h$.

Algorithm / Steps:

- Create abstract Shape.
- Declare area and volume methods.
- Create Cylinder subclass.
- Implement formulas.
- Display results.

Program:

```
abstract class Shape{abstract double area();abstract double volume();}
class Cylinder extends Shape{
    double r,h;Cylinder(double r,double h){this.r=r;this.h=h;}
    double area(){return 2*Math.PI*r*(r+h);}
    double volume(){return Math.PI*r*r*h;}
    public static void main(String[]x){Cylinder c=new
    Cylinder(7,10);System.out.println("Area="+c.area());System.out.println("Volume="+c.volume());}
}
```

Expected Output:

```
For r=7, h=10: Area ≈ 747.70, Volume ≈ 1539.38
```

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Write a Node.js program to demonstrate synchronous and asynchronous file operations and compare their execution.

Aim:

To demonstrate synchronous and asynchronous fs operations.

Theory:

Synchronous methods block execution until completion; asynchronous methods use callbacks and allow the event loop to continue.

Algorithm / Steps:

- Perform writeFileSync/readFileSync.
- Perform writeFile/readFile.

- Print messages before/after.
- Observe execution order.

Program:

```
const fs=require('fs');
console.log('Sync start');
fs.writeFileSync('sync.txt','Sync data');
console.log(fs.readFileSync('sync.txt','utf8'));
console.log('Sync end');

console.log('Async start');
fs.writeFile('async.txt','Async data',(e)=>{
  if(e)throw e;
  fs.readFile('async.txt','utf8',(e,d)=>{if(e)throw e;console.log(d);});
});
console.log('Async end');
```

Expected Output:

Typical order: Sync start → data → Sync end; Async start → Async end → data.

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 25

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a Java program that defines an interface Calculator containing methods add() and subtract() and implements it in class SimpleCalc.

Aim:

To demonstrate an interface and its implementation.

Theory:

An interface declares methods. SimpleCalc implements Calculator and provides method bodies.

Algorithm / Steps:

- Declare Calculator.
- Implement in SimpleCalc.
- Define add and subtract.
- Create object.
- Call methods.

Program:

```
interface Calculator{int add(int a,int b);int subtract(int a,int b);}
class SimpleCalc implements Calculator{
    public int add(int a,int b){return a+b;}
    public int subtract(int a,int b){return a-b;}
    public static void main(String[]x){SimpleCalc c=new
SimpleCalc();System.out.println(c.add(20,5));System.out.println(c.subtract(20,5));}
}
```

Expected Output:

```
25
15
```

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Write a Node.js program to perform directory management operations such as creating, listing, renaming and deleting directories. Also demonstrate JSON parsing and writing structured JSON data.

Aim:

To manage directories and structured JSON data using Node.js.

Theory:

fs.mkdirSync, readdirSync, renameSync and rmSync manage directories. JSON.stringify converts objects to text and JSON.parse converts text to objects.

Algorithm / Steps:

- Create directory.

- List entries.
- Rename directory.
- Write JSON.
- Parse JSON and delete directory.

Program:

```
const fs=require('fs');
fs.mkdirSync('dataDir',{recursive:true});
console.log(fs.readdirSync('.'));
fs.renameSync('dataDir','newDataDir');
const obj={name:'Amit',marks:85};
fs.writeFileSync('student.json',JSON.stringify(obj,null,2));
const data=JSON.parse(fs.readFileSync('student.json','utf8'));
console.log(data);
fs.rmSync('newDataDir',{recursive:true,force:true});
```

Expected Output:

```
Directory operations complete; parsed JSON object is displayed as { name: 'Amit', marks: 85 }.
```

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 26

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Define a class Student with data members student_name, student_rollno, total_lectures, attended_lectures. Create an object of Student. If attendance percentage is less than 75%, throw a user-defined exception "Student is Not Eligible for Exam", otherwise display student details.

Aim:

To validate student attendance using a user-defined exception.

Theory:

Attendance percentage = $\frac{\text{attended lectures}}{\text{total lectures}} \times 100$. A custom exception is thrown when it is below 75%.

Algorithm / Steps:

- Create Student class.
- Create custom exception.
- Calculate attendance.
- Throw if below 75%.
- Otherwise display details.

Program:

```
class AttendanceException extends Exception{AttendanceException(String m){super(m);}}
class Student{
    String student_name;int student_rollno,total_lectures,attended_lectures;
    Student(String n,int r,int t,int a){student_name=n;student_rollno=r;total_lectures=t;attended_lectures=a;}
    void check()throws AttendanceException{
        double p=attended_lectures*100.0/total_lectures;
        if(p<75)throw new AttendanceException("Student is Not Eligible for Exam");
        System.out.println(student_name+" "+student_rollno+" Attendance="+p+"%");
    }
    public static void main(String[]x){Student s=new
    Student("Yash",21,100,82);try{s.check();}catch(AttendanceException e){System.out.println(e.getMessage());}}
}
```

Expected Output:

```
Yash 21 Attendance=82.0%
```

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Write a Node.js program using the fs module to copy, rename and delete files asynchronously. Display appropriate success and error messages.

Aim:

To perform asynchronous file copy, rename and delete operations.

Theory:

fs.copyFile(), rename() and unlink() are asynchronous callback-based methods.

Algorithm / Steps:

- Create source file.
- Copy it.
- Rename copy.
- Delete renamed file.
- Handle errors.

Program:

```
const fs=require('fs');
fs.writeFile('source.txt','Sample data',err=>{
  if(err) return console.log(err.message);
  fs.copyFile('source.txt','copy.txt',err=>{
    if(err) return console.log(err.message); console.log('Copied');
    fs.rename('copy.txt','renamed.txt',err=>{
      if(err) return console.log(err.message); console.log('Renamed');
      fs.unlink('renamed.txt',err=>console.log(err?'Delete error': 'Deleted'));
    });
  });
});
```

Expected Output:

```
Copied
Renamed
Deleted
```

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 27

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a program to define class Person with data members as Personname, Aadhar no, Panno. Accept information for 5 objects and display appropriate information (use this keyword).

Aim:

To use this keyword and an array of five Person objects.

Theory:

this refers to the current object and is used to distinguish instance variables from constructor parameters.

Algorithm / Steps:

- Define Person members.
- Use parameterized constructor with this.
- Create five objects.
- Store them in array.
- Display information.

Program:

```
import java.util.*;
class Person{
    String Personname,Aadharno,Panno;
    Person(String n,String a,String p){this.Personname=n;this.Aadharno=a;this.Panno=p;}
    void show(){System.out.println(Personname+" "+Aadharno+" "+Panno);}
    public static void main(String[]x){
        Scanner s=new Scanner(System.in);Person[] p=new Person[5];
        for(int i=0;i<5;i++)p[i]=new Person(s.next(),s.next(),s.next());
        for(Person q:p)q.show();
    }
}
```

Expected Output:

```
Five Person records are accepted and displayed with name, Aadhar number and PAN number.
```

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Write a Node.js program to read a JSON file, modify selected properties of the JSON object, and write the updated JSON data back to the file using the fs module

Aim:

To read, modify and write JSON data using Node.js fs.

Theory:

JSON.parse converts file text to an object; after modification JSON.stringify converts it back to JSON text.

Algorithm / Steps:

- Read JSON file.
- Parse JSON.
- Modify selected properties.
- Stringify object.
- Write updated JSON.

Program:

```
const fs=require('fs');
fs.readFile('student.json','utf8',(err,data)=>{
  if(err)throw err;
  const obj=JSON.parse(data);
  obj.name='Rahul'; obj.marks=90;
  fs.writeFile('student.json',JSON.stringify(obj,null,2),err=>{
    if(err)throw err; console.log('JSON updated successfully');
  });
});
```

Expected Output:

```
JSON updated successfully; selected properties contain the new values.
```

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 28

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Define a class MyDate(Day, Month, year) with methods to accept and display a MyDate object. Accept date as dd,mm,yyyy. Throw user defined exception "InvalidDateException" if the date is invalid.

Aim:

To validate a date using a user-defined InvalidDateException.

Theory:

A valid date must have a valid month/day combination; Java LocalDate can validate the actual calendar date.

Algorithm / Steps:

- Create exception class.
- Create MyDate.
- Read dd, mm, yyyy.
- Validate with LocalDate.
- Throw exception if invalid.
- Display valid date.

Program:

```
import java.time.*;
import java.util.*;
class InvalidDateException extends Exception{InvalidDateException(String m){super(m);}}
class MyDate{
    int day,month,year;
    MyDate(int d,int m,int y){day=d;month=m;year=y;}
    void check()throws InvalidDateException{
        try{LocalDate.of(year,month,day);System.out.println("Valid Date: "+day+"-"+month+"-"+year);}
        catch(DateTimeException e){throw new InvalidDateException("InvalidDateException");}
    }
    public static void main(String[]x){
        Scanner s=new Scanner(System.in);MyDate d=new MyDate(s.nextInt(),s.nextInt(),s.nextInt());
        try{d.check();}catch(InvalidDateException e){System.out.println(e.getMessage());}
    }
}
```

Expected Output:

```
Input 29 2 2021 → InvalidDateException. Input 28 2 2021 → Valid Date: 28-2-2021
```

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Create a Node.js program to list files in a specified directory, filter files based on their extensions (such as .txt, .json and .js), and display the matching file names.

Aim:

To list and filter directory files by extension.

Theory:

fs.readdir() reads directory entries. path.extname() returns the extension of a file name.

Algorithm / Steps:

- Accept directory.
- Read files.
- Get extension.
- Compare with .txt/.json/.js.
- Display matching names.

Program:

```
const fs=require('fs');const path=require('path');
const dir='.';
fs.readdir(dir, (err, files)=>{
  if(err)throw err;
  files.filter(f=>['.txt','.json','.js'].includes(path.extname(f)))
    .forEach(f=>console.log(f));
});
```

Expected Output:

Only files ending in .txt, .json or .js are displayed.

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 29

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Write a program to accept a number from the user, if number is zero then throw user defined exception "Number is 0" otherwise check whether no is prime or not (Use static keyword)

Aim:

To check prime status using a static method and a user-defined exception.

Theory:

A static method belongs to the class and can be called without creating an object. Zero is rejected through a custom exception.

Algorithm / Steps:

- Create exception.
- Create static prime method.
- Read number.
- Throw exception for zero.
- Check divisibility and display.

Program:

```
import java.util.*;
class ZeroException extends Exception{ZeroException(String m){super(m);}}
class PrimeCheck{
    static boolean isPrime(int n){if(n<2)return false;for(int i=2;i*i<=n;i++)if(n%i==0)return false;return true;}
    public static void main(String[]x){
        Scanner s=new Scanner(System.in);int n=s.nextInt();
        try{if(n==0)throw new ZeroException("Number is 0");System.out.println(isPrime(n)?"Prime":"Not Prime");}
        catch(ZeroException e){System.out.println(e.getMessage());}
    }
}
```

Expected Output:

Input 0 → Number is 0. Input 17 → Prime. Input 18 → Not Prime.

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Write a Node.js program to demonstrate file statistics using fs.stat(), including file size, file type, creation/access/modification information, and whether the path represents a file or directory.

Aim:

To obtain file or directory statistics using fs.stat().

Theory:

fs.stat() returns metadata such as size, birthtime, atime, mtime and methods like isFile() and isDirectory().

Algorithm / Steps:

- Accept path.
- Call fs.stat().
- Display size.
- Display time information.
- Check file/directory type.

Program:

```
const fs=require('fs');
fs.stat('student.json',(err,stats)=>{
  if(err)return console.log(err.message);
  console.log('Size:',stats.size);
  console.log('Created:',stats.birthtime);
  console.log('Accessed:',stats.atime);
  console.log('Modified:',stats.mtime);
  console.log('Is File:',stats.isFile());
  console.log('Is Directory:',stats.isDirectory());
});
```

Expected Output:

Displays file size, creation/access/modification timestamps, Is File: true and Is Directory: false for a file.

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.

CS-306-MJ-P — PRACTICAL EXAM SLIP 30

Core Java and Web Technology-I | Lab Book Format

Q1 — 15 Marks

Define a class MyNumber having one private integer data member. Write a default constructor initialize it to 0 and another constructor to initialize it to a value. Write methods isNegative, isPositive, isOdd, isEven. Use command line argument to pass a value to the object and perform the above operations.

Aim:

To demonstrate constructors, private data and methods using a command-line argument.

Theory:

The private data member is initialized by constructors. Command-line arguments are Strings and are converted using Integer.parseInt().

Algorithm / Steps:

- Create private integer.
- Create default constructor.
- Create parameterized constructor.
- Convert args[0] to int.
- Call isNegative/isPositive/isOdd/isEven.

Program:

```
class MyNumber{
    private int n;
    MyNumber(){n=0;}
    MyNumber(int n){this.n=n;}
    void isNegative(){System.out.println("Negative: "+(n<0));}
    void isPositive(){System.out.println("Positive: "+(n>0));}
    void isOdd(){System.out.println("Odd: "+(n%2!=0));}
    void isEven(){System.out.println("Even: "+(n%2==0));}
    public static void main(String[]args){
        if(args.length==0){System.out.println("Pass a number as command line argument");return;}
        MyNumber m=new MyNumber(Integer.parseInt(args[0]));
        m.isNegative();m.isPositive();m.isOdd();m.isEven();
    }
}
```

Expected Output:

```
Command: java MyNumber 7
Negative: false
Positive: true
Odd: true
Even: false
```

Result:

The program was executed successfully and the required output was obtained.

Q2 — 15 Marks

Create a Node.js mini file-management application using the fs and path modules that allows the user to create, read, update, rename and delete text files through a menu-driven interface.

Aim:

To create a menu-driven file management application using fs and path.

Theory:

readline accepts menu input; fs performs create/read/update/rename/delete and path builds file paths.

Algorithm / Steps:

- Create readline interface.
- Display menu.
- Implement create/read/update/rename/delete.
- Use path.join() for file paths.
- Repeat until Exit.

Program:

```
const fs=require('fs'),path=require('path'),readline=require('readline');
const rl=readline.createInterface({input:process.stdin,output:process.stdout});
function menu(){
  console.log('\n1.Create 2.Read 3.Update 4.Rename 5.Delete 6.Exit');
  rl.question('Choice: ',c=>{
    if(c==='1')fs.writeFileSync(path.join(__dirname,'file.txt'),'New file');
    else if(c==='2')console.log(fs.readFileSync(path.join(__dirname,'file.txt'),'utf8'));
    else if(c==='3')fs.appendFileSync(path.join(__dirname,'file.txt'),'\nUpdated');
    else if(c==='4')fs.renameSync('file.txt','renamed.txt');
    else if(c==='5')fs.unlinkSync('renamed.txt');
    else if(c==='6'){rl.close();return;}
    console.log('Operation completed');menu();
  });
}
menu();
```

Expected Output:

Menu appears with Create, Read, Update, Rename, Delete and Exit; selected file operation is performed with a success message.

Result:

The webpage/program was created successfully and the required operation was performed.

Q3 — Viva

Prepare the concepts, syntax, program logic and output of Q1 and Q2.